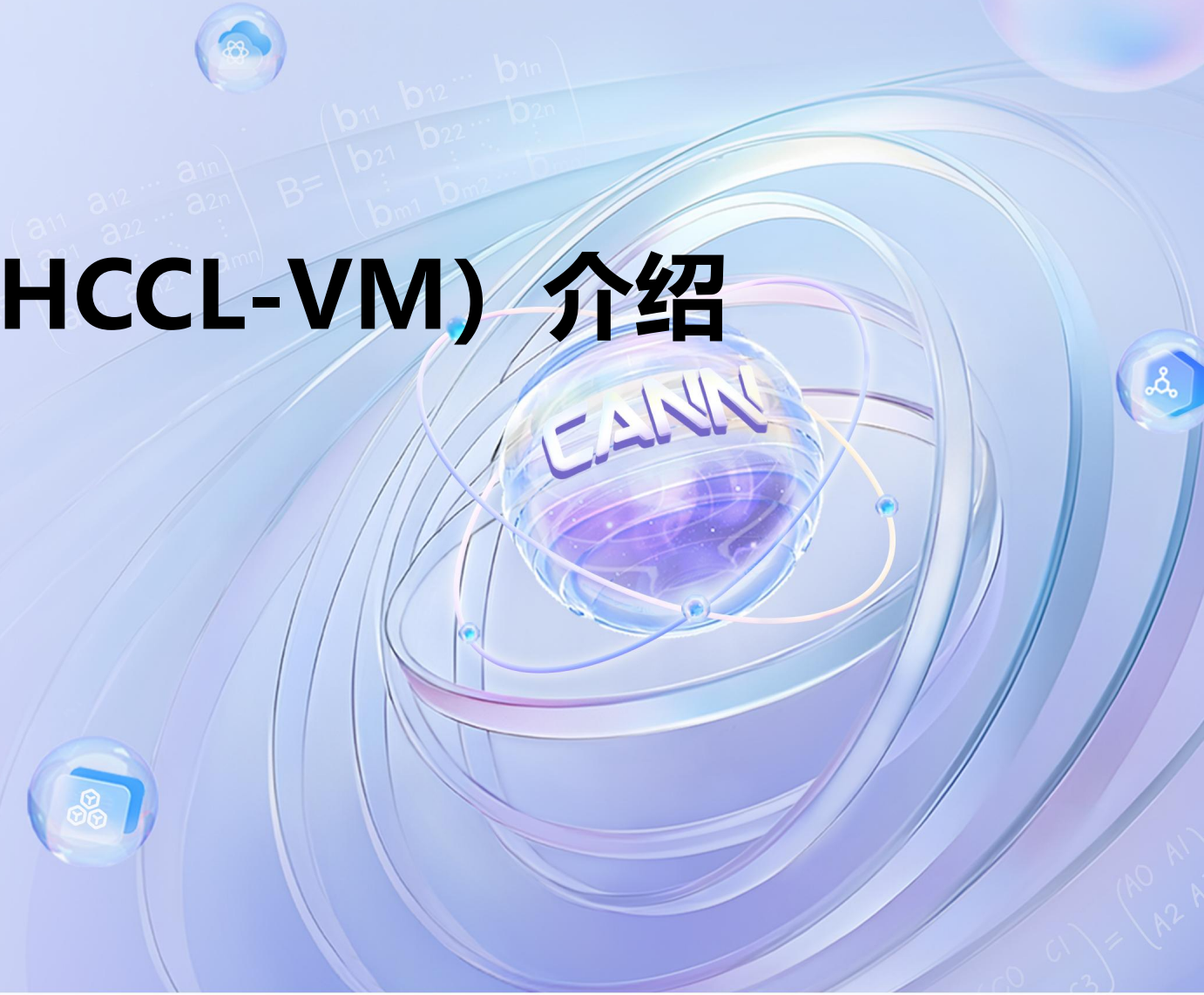
HCCL北极星平台（HCCL-VM）介绍



CANN

修订记录

版本号	修订日期	修订人	修订页码	修订内容摘要
V5.0	2026/7/30	张兴平	全部	初版内容

目录

Part 1 背景

Part 2 HCCL北极星平台总体架构

Part 3 模拟运行工具

Part 4 算法分析工具

Part 5 工具使用

背景：大模型时代 HCCL 验证范式的全新演进

大模型时代物理验证的新命题

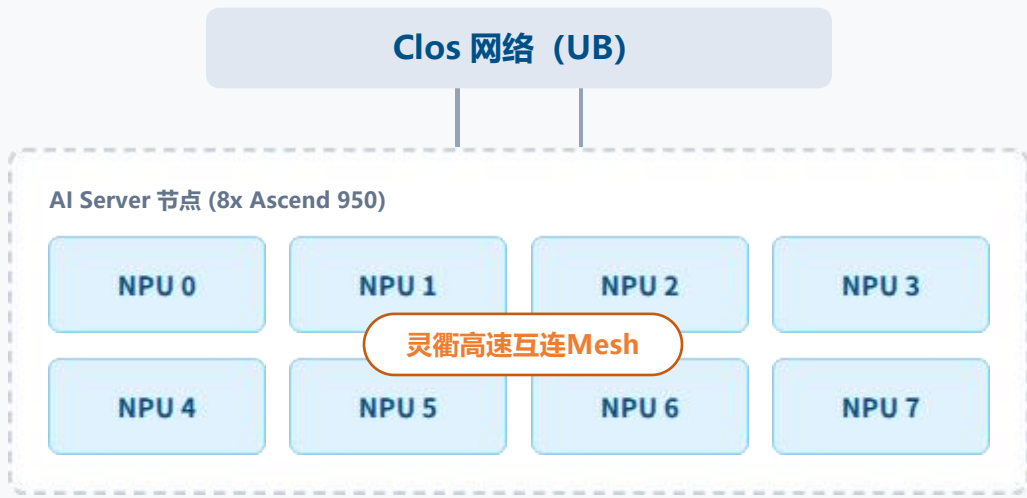
- **通信成瓶颈枢纽：**智算集群大模型（如 MoE、All-to-All）训练中，HCCL 的微小抖动在千卡规模会被指数级放大。
- **验证成本攀升：**物理上板验证依赖昂贵的集群资源，排队时间长，难以满足当前敏捷迭代的研发需求
- **全链路定界复杂：**集群中，软硬件耦合极深，快速区分是“算法逻辑 Bug”、“内存并发踩踏”还是“底层网络抖动”，极具挑战。

北极星验证新范式

秉持前置排雷理念，将必须上机才能发现的海量逻辑问题，拦截在纯软分析阶段，释放稀缺的真机算力潜能。

0 物理 NPU 占用　本地一键拉起　偶发转离线必现

典型 Ascend 950 多卡集群物理拓扑



⚠ 硬件排障黑盒困境

多卡并发时，极小的算法逻辑越界极易被复杂的 **UB/ROCE** 等物理网卡与路由拓扑掩盖。传统模式需长周期 Dump 重放，定界效率极低。

代码仓：https://gitcode.com/cann/hcomm/tree/master/test/hccl_vm

目录

Part 1 背景

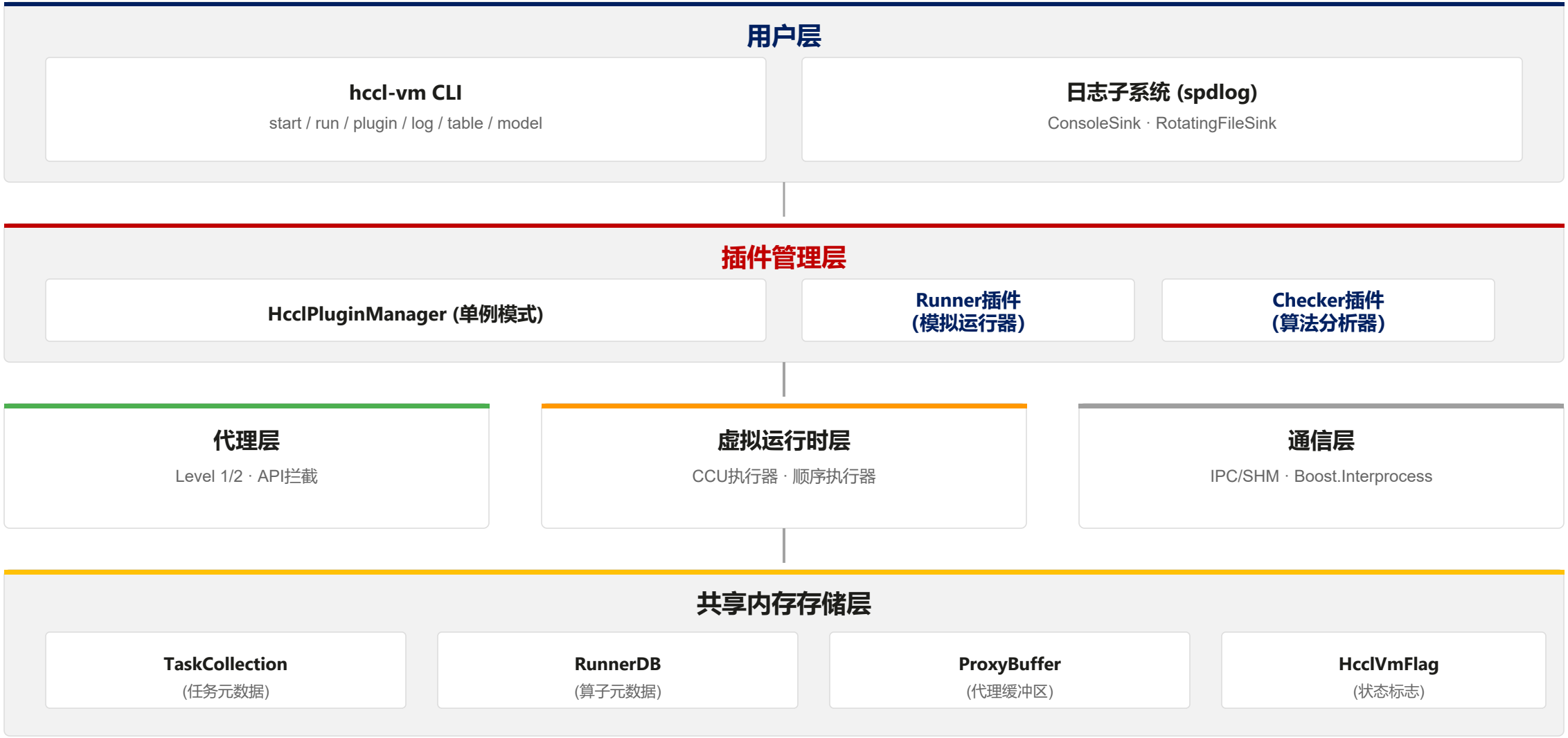
Part 2 HCCL北极星平台总体架构

Part 3 模拟运行工具

Part 4 算法分析工具

Part 5 工具使用

北极星工具（HCCL-VM）： 总体架构



目录

Part 1 背景

Part 2 HCCL北极星平台总体架构

Part 3 模拟运行工具

Part 4 算法分析工具

Part 5 工具使用

模拟运行器原理

核心机制

通过 LD_PRELOAD 动态库拦截技术，透明捕获 HCCL API 调用，提取任务元数据，在虚拟运行时中模拟执行并验证正确性。

执行流程

01
API拦截阶段
Level 2 算子级拦截
Level 1 任务级拦截

02
任务收集阶段
元数据写入共享内存
TaskVentilator 分发

03
模拟执行阶段
SqeuentialExecutor
多类型任务分发执行

关键技术点

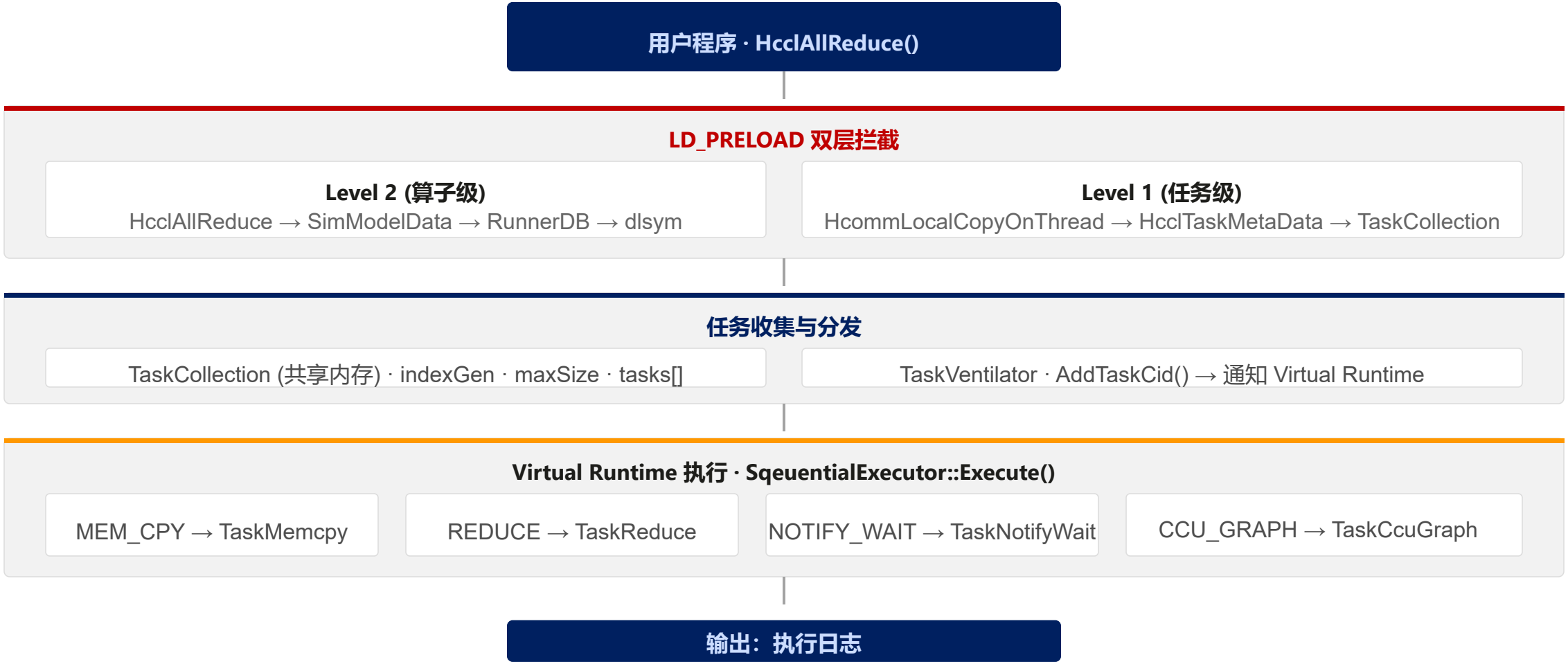
技术	实现方式	优势
LD_PRELOAD 动态库拦截	libhccl_proxy_level1/2.so	无侵入、透明拦截
共享内存	Boost.Interprocess	高效进程间通信
柔性数组	TaskCollection::tasks[]	可变数量任务存储
工厂模式	CcuExecutorFactory	易于扩展新指令类型

运行模式

Runner (HCCL_VM_MODE=RUNNER) → 指令级模拟执行

Hybrid (HCCL_VM_MODE=HYBRID) → 模拟执行 + 校验

模拟运行器： 执行流程


用户程序 · HcclAllReduce()
LD_PRELOAD 双层拦截
Level 2 (算子级)
HcclAllReduce → SimModelData → RunnerDB → dlsym
Level 1 (任务级)
HcommLocalCopyOnThread → HcclTaskMetaData → TaskCollection
任务收集与分发
TaskCollection (共享内存) · indexGen · maxSize · tasks[]
TaskVentilator · AddTaskCid() → 通知 Virtual Runtime
Virtual Runtime 执行 · SqeuentialExecutor::Execute()
MEM_CPY → TaskMemcpy
REDUCE → TaskReduce
NOTIFY_WAIT → TaskNotifyWait
CCU_GRAPH → TaskCcuGraph
输出：执行日志

目录

Part 1 背景

Part 2 HCCL北极星平台总体架构

Part 3 模拟运行工具

Part 4 算法分析工具

Part 5 工具使用

算法分析器原理

核心洞察

- **动态时态捕获：** 摒弃局限的静态源码推演，通过虚拟环境底层静默劫持，精准获取真实运行时的 API 调用序列。
- **自动化分析链路：** 独创“输入捕获 ➔ 拓扑成图 ➔ 基础排雷 ➔ 双路校验”的标准化、端到端验证流水线。

1

输入捕获

底层静默劫持真实 HCCL API，获取真实调用任务，重组为多组 Task Queues。

2

底座构建

提取通信与同步原语，建立 Post/Wait 配对与跨队列依赖，生成全局任务图。

3

前置排雷

执行基础结构校验：排查环形依赖、悬挂配对关系。回答“图能不能走通”。

4

双路高阶校验

支线 A (语义)：结果图案是否符合协议。

支线 B (内存)：执行中资源是否被踩坏。

设计优势：系统关注指令因果关系和拓扑依赖，本质是对任务图做逻辑推演，而非真实申请内存去做物理执行。

CANN

算法分析器：语义校验引擎原理

核心洞察

- **轻量化推演引擎：** 不需要进行真实浮点运算与物理周期仿真，通过数据结构定义不同任务，实现极低资源开销的逻辑推演。
- **血缘“染色”模型：** 跨越数值表象，通过追踪数据区间流向，高效且精确地验证最终输出图案与算子协议的一致性。

动态传播规则

1. 覆盖式传播 (Override) - 模拟 Copy / Read / Write



2. 归约式合并 (Reduce) - 模拟 Sum / Max / Min



算子协议标准图案



CANN

算法分析器：内存冲突校验引擎原理

核心洞察

- **全景过程安全防线：** 作为语义校验的严谨补充，专职拦截执行过程中的中间状态破坏与隐藏内存污染隐患。
- **高阶并发裁决机制：** 确立“先判并发偏序，再判地址重叠”的判别准则，精准捕获并发重叠写，彻底消除偶发性 Bug。

核心算法：三步曲

1 片段化 (Fragmentation)



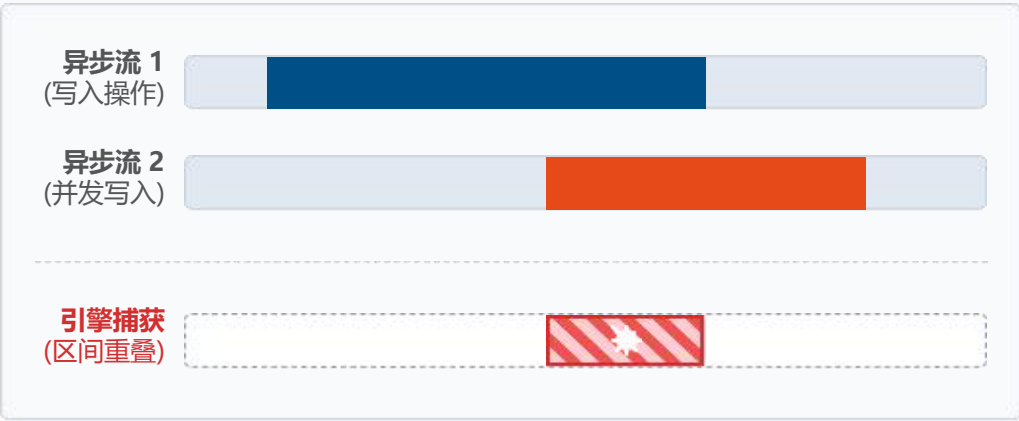
2 并发过滤 (Concurrency Pruning)



工程化原则： 设计优先级为 并发语义正确 > 结果可解释 > 主流规模性能可接受。

【步骤 3 放大】并发区间碰撞

仅对保留下来的“并发片段”进行比较。如果两个并发片段访问了重叠的内存区间，且**至少包含一个写操作**，即判定为冲突。



目录

Part 1 背景

Part 2 HCCL北极星平台总体架构

Part 3 模拟运行工具

Part 4 算法分析工具

Part 5 工具使用

北极星工具使用介绍

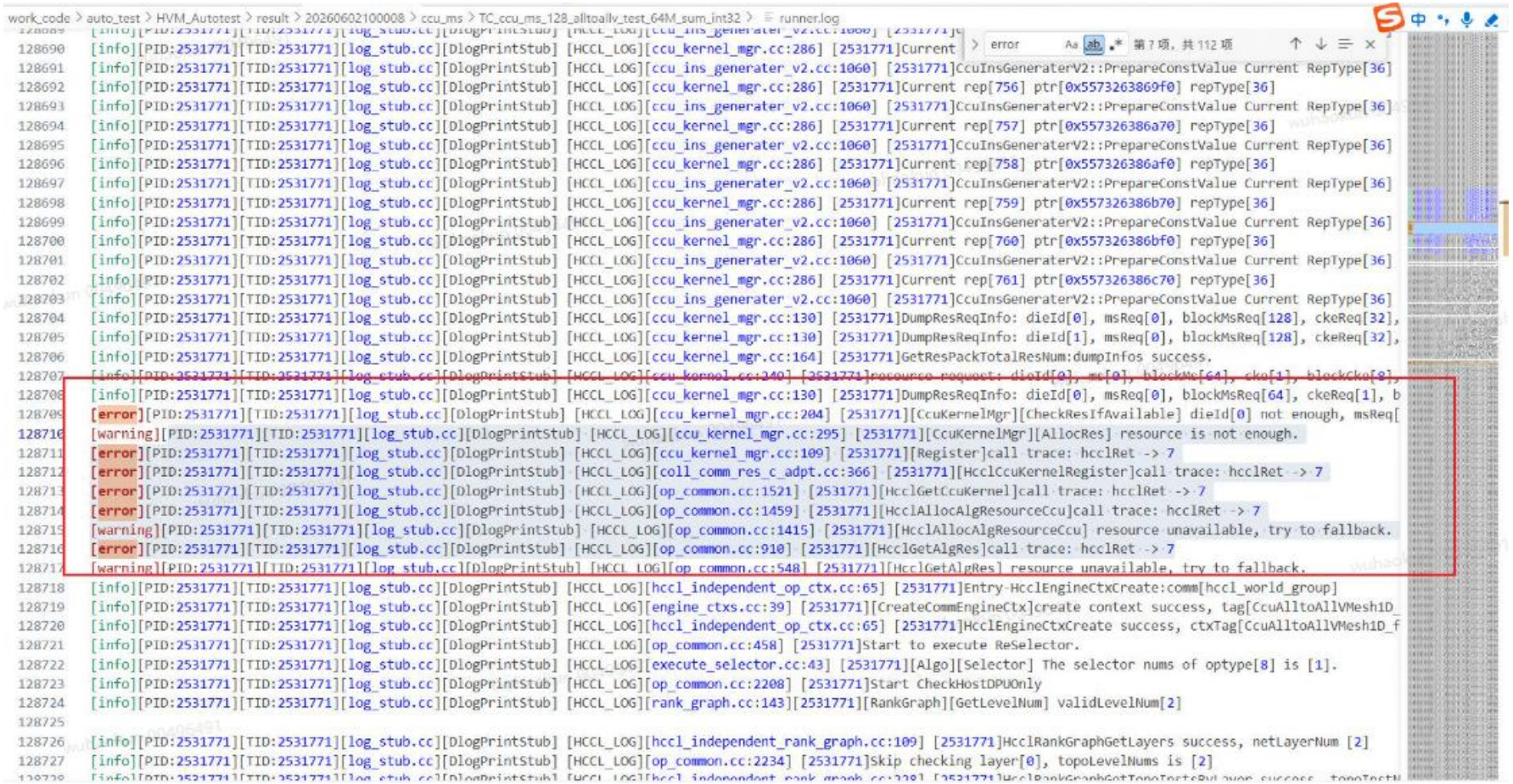
子命令	源文件	职责
log	log_subcmd.cpp	查询/设置日志等级
mock-comm	mock_comm_subcmd.cpp	为单个算子建立通信域
model	model_subcmd.cpp	列出建模文件
plugin	plugin_subcmd.cpp	插件安装/卸载/运行/列表
reset	reset_subcmd.cpp	清空数据库表、重置仿真环境
start	start_subcmd.cpp	启动仿真环境并进入交互 shell
table	table_subcmd.cpp	展示/更新仿真数据库表

plugin	install @checker/@runner	安装checker、runner插件
	uninstall @checker/@runner	卸载checker、runner插件
	run @checker/@runner	执行checker、runner插件

详细安装部署和使用指导见代码仓：https://gitcode.com/cann/hcomm/tree/master/test/hccl_vm

CANN

北极星工具实践：常见问题提前拦截

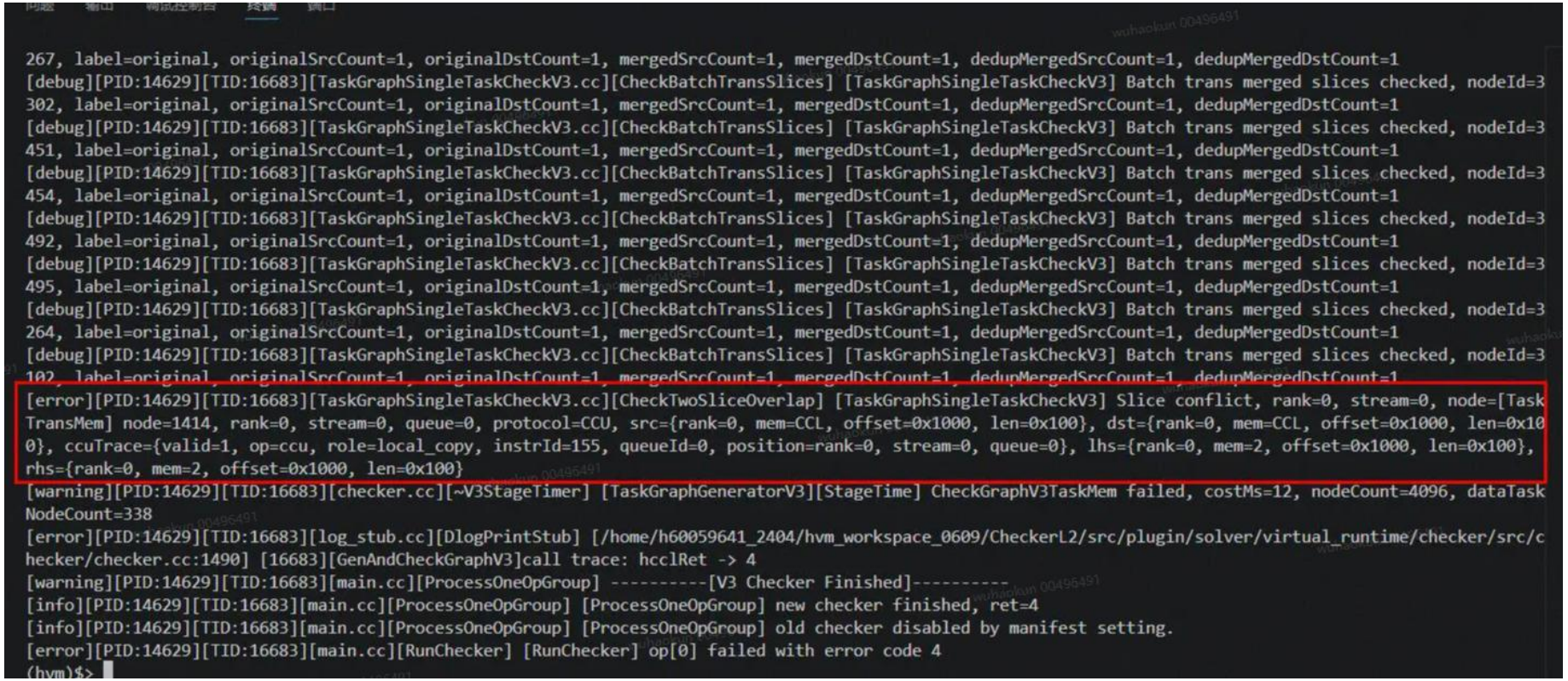


模拟运行器报错：
CCU资源不足

https://gitcode.com/cann

CANN

北极星工具实践：算法逻辑错误提前识别



算法分析器报错：
内存冲突校验不通过

CANN

Thank you.

社区愿景：打造开放易用、技术领先的AI算力新生态

社区使命：使能开发者基于CANN社区自主研究创新，构筑根深叶茂、跨产业协同共享共赢的CANN生态

Vision: Building an Open, Easy-to-Use, and Technology-leading AI Computing Ecosystem

Mission: Enable developers to independently research and innovate based on the CANN community and build a win-win CANN ecosystem with deep roots and cross-industry collaboration and sharing.

上CANN社区获取干货

关注CANN公众号获取资讯

https://gitcode.com/cann

CANN